# FastAPI & REST API

Complete Concept Guide • Beginner to Practical Backend Development

## What you will learn

- REST principles, HTTP methods, status codes, request/response lifecycle
- FastAPI setup, routing, validation, Pydantic, dependency injection
- CRUD APIs, database integration, SQLAlchemy, authentication and security
- Error handling, testing, documentation, project structure and deployment
- A practical learning-app API example and interview revision questions

Prepared as a hands-on study reference. Examples use Python and FastAPI.

# 1. API and REST API fundamentals

**API (Application Programming Interface)** is a contract that allows two software components to communicate. A frontend can call a backend API to request data or ask it to perform an action.

**Example:** A learning app frontend sends a request to the backend for a list of courses. The backend validates the request, reads the database, and returns JSON.

## What is REST?

REST (Representational State Transfer) is an architectural style for designing networked APIs. A REST API commonly exposes resources through URLs and uses HTTP methods to operate on those resources.

| Term | Meaning | Example |
|---|---|---|
| Resource | An entity or collection managed by the API | courses, students, orders |
| Endpoint | A URL path for accessing a resource | /api/courses/12 |
| Request | Message sent by the client | GET /api/courses |
| Response | Message returned by the server | 200 OK with JSON |
| JSON | Common data format exchanged by APIs | {"id":12,"name":"Python"} |

## REST constraints and good practices

• Client-server separation: frontend and backend have separate responsibilities.

• Statelessness: each request contains the information needed to process it; the server does not rely on hidden client session state.

• Cacheability: responses can indicate whether they may be cached.

• Uniform interface: consistent resource URLs, methods, representations and status codes.

• Layered system: clients do not need to know whether they communicate through proxies, gateways or the application server.

• Code-on-demand is optional and rarely central to ordinary JSON APIs.

REST is an architectural style, not a programming language or framework. FastAPI is a Python framework that can be used to build REST APIs.

# 2. HTTP request and response

## Request structure

```
GET /api/courses/12 HTTP/1.1
Host: localhost:8000
Accept: application/json
Authorization: Bearer <token>
```

A request generally contains a method, path, headers, and sometimes a body. Query parameters are placed after ? in the URL.

## Path parameters vs query parameters vs request body

| Type | Purpose | Example |
|---|---|---|
| Path parameter | Identifies a particular resource | /courses/12 |
| Query parameter | Filters, sorts or paginates | /courses?level=beginner&page=1 |

| Type | Purpose | Example |
|---|---|---|
| Request body | Sends structured data to create/update | JSON body with course fields |
| Header | Carries metadata or credentials | Content-Type, Authorization |

### Common HTTP methods

| Method | Typical purpose | Idempotent? |
|---|---|---|
| GET | Read a resource | Yes |
| POST | Create a resource or trigger an operation | Usually no |
| PUT | Replace a resource representation | Yes |
| PATCH | Partially update a resource | Not guaranteed; often designed to be |
| DELETE | Delete a resource | Yes in intended effect |

Idempotent means repeating the same request has the same intended effect as making it once. A repeated POST can create duplicates unless the API uses an idempotency strategy.

### Status codes to know

| Code | Meaning | Typical use |
|---|---|---|
| 200 | OK | Successful read/update |
| 201 | Created | Resource created |
| 202 | Accepted | Work accepted for asynchronous processing |
| 204 | No Content | Successful operation with no response body |
| 400 | Bad Request | Malformed or invalid request |
| 401 | Unauthorized | Missing or invalid authentication |
| 403 | Forbidden | Authenticated but not allowed |
| 404 | Not Found | Resource does not exist |
| 409 | Conflict | Duplicate or state conflict |
| 422 | Unprocessable Content | Request data fails validation |
| 429 | Too Many Requests | Rate limit exceeded |
| 500 | Internal Server Error | Unexpected server failure |
| 503 | Service Unavailable | Temporarily unavailable |

# 3. What is FastAPI?

FastAPI is a modern Python web framework for building APIs. It uses Python type hints, Starlette for core web capabilities, and Pydantic for data parsing and validation. It can generate OpenAPI documentation automatically.

### Why developers use it

• Type-hint based request validation and clear error responses.

• Automatic interactive API documentation at /docs and /redoc.

• Async support for I/O-bound operations.

• Dependency injection for reusable logic such as database sessions and authentication.

• Good developer experience and strong editor support.

• Suitable for small services as well as larger, modular backend applications when designed well.

### FastAPI vs Flask vs Django

| Framework | Main idea | Common fit |
|---|---|---|
| FastAPI | API-first, typed validation, OpenAPI | REST APIs and async services |
| Flask | Small, flexible web framework | Small services and custom architectures |
| Django | Full-featured web framework with ORM/admin | Database-heavy web applications |

FastAPI does not automatically make an application scalable or secure. Architecture, database design, deployment, monitoring and security still matter.

## 4. Environment setup

Use a supported Python version and create a project-specific virtual environment. Run these commands in PowerShell from your project folder:

```
mkdir learning_api
cd learning_api
py -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install "fastapi[standard]" sqlalchemy pymysql python-dotenv
```

If PowerShell blocks activation, use an approved execution-policy approach for your machine, or run the environment's Python directly. Do not disable security controls globally without understanding the impact.

### Create main.py

```
from fastapi import FastAPI

app = FastAPI(title="Learning API", version="1.0.0")

@app.get("/")
def home():
    return {"message": "Learning API is running"}

@app.get("/health")
def health():
    return {"status": "healthy"}

fastapi dev main.py
```

Open http://127.0.0.1:8000/ for the root response, /docs for Swagger UI, and /redoc for alternative API documentation.

## 5. Routes, parameters and responses

### Route decorators

```
@app.get("/courses")
def list_courses():
    return [{"id": 1, "name": "Python"}]

@app.get("/courses/{course_id}")
def get_course(course_id: int):
    return {"course_id": course_id}
```

The decorator maps an HTTP method and URL path to a Python function. FastAPI converts and validates typed path parameters.

### Query parameters

```
@app.get("/courses")
def list_courses(level: str | None = None, page: int = 1, limit: int = 10):
    return {
        "level": level,
        "page": page,
        "limit": limit
    }
```

Example: GET /courses?level=beginner&page;=2&limit;=10

### Response models

```
from pydantic import BaseModel

class CourseOut(BaseModel):
    id: int
    name: str
    level: str

@app.get("/courses/{course_id}", response_model=CourseOut)
def get_course(course_id: int):
    return {"id": course_id, "name": "Python", "level": "Beginner"}
```

A response model documents and validates the output shape. It can also prevent internal fields from being exposed.

## 6. Pydantic and request validation

Pydantic models define the expected shape and types of incoming data. FastAPI uses them to parse and validate JSON request bodies.

```
from pydantic import BaseModel, Field

class CourseCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    description: str | None = None
    duration_hours: int = Field(gt=0, le=500)
    level: str

@app.post("/courses", status_code=201)
def create_course(course: CourseCreate):
    return {
        "message": "Course created",
        "course": course
    }
```

Example JSON body:

```
{
  "name": "Python Fundamentals",
  "description": "Learn Python from basics",
  "duration_hours": 30,
  "level": "beginner"
}
```

If duration_hours is zero or the name is too short, validation fails and FastAPI returns a structured 422 response. Validation should be treated as a boundary check, not a replacement for business rules.

## 7. Build CRUD APIs

CRUD means Create, Read, Update and Delete. A course resource can use the following endpoint design:

| Operation | HTTP method | Endpoint |
|---|---|---|
| Create | POST | /courses |

| Operation | HTTP method | Endpoint |
|---|---|---|
| Read all | GET | /courses |
| Read one | GET | /courses/{course_id} |
| Replace | PUT | /courses/{course_id} |
| Partial update | PATCH | /courses/{course_id} |
| Delete | DELETE | /courses/{course_id} |

### In-memory learning example

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
courses = {}
next_id = 1

class CourseCreate(BaseModel):
    name: str
    level: str

@app.post("/courses", status_code=201)
def create_course(data: CourseCreate):
    global next_id
    course = {"id": next_id, **data.model_dump()}
    courses[next_id] = course
    next_id += 1
    return course

@app.get("/courses")
def get_courses():
    return list(courses.values())

@app.get("/courses/{course_id}")
def get_course(course_id: int):
    course = courses.get(course_id)
    if course is None:
        raise HTTPException(status_code=404, detail="Course not found")
    return course

@app.delete("/courses/{course_id}", status_code=204)
def delete_course(course_id: int):
    if course_id not in courses:
        raise HTTPException(status_code=404, detail="Course not found")
    del courses[course_id]
    return
```

This is a teaching example only: in-memory data disappears when the process restarts and is not suitable for multi-worker production use.

# 8. Database integration and SQLAlchemy

A real application stores persistent data in a database such as PostgreSQL or MySQL. SQLAlchemy is a Python SQL toolkit and ORM. The ORM maps Python classes to database tables and helps build queries.

### Important concepts

• Engine: manages connectivity to the database.

• Session: unit of work used to query and persist data.

• Model: Python class mapped to a database table.

• Schema: Pydantic model used for input/output validation.

• Migration: controlled versioned change to database structure; Alembic is commonly used.

• Transaction: group of database operations that commit together or roll back on failure.

### Simple SQLAlchemy 2.x model

```
from sqlalchemy import create_engine, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, sessionmaker

DATABASE_URL = "mysql+pymysql://user:password@localhost:3306/learning_db"

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):
    pass

class Course(Base):
    __tablename__ = "courses"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    level: Mapped[str] = mapped_column(String(30))

Base.metadata.create_all(bind=engine)
```

For production, keep credentials in environment variables or a secret manager, not in source code. Use migrations rather than relying on create_all for schema evolution.

### Database dependency pattern

```
from collections.abc import Generator

def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Inject the session into route functions with Depends(get_db). This ensures each request gets a managed session and the session is closed afterward.

## 9. Dependency injection

Dependency injection (DI) lets FastAPI provide shared resources or checks to route functions. Common dependencies include database sessions, authenticated users, configuration and pagination.

```
from fastapi import Depends

@app.get("/private-courses")
def private_courses(db = Depends(get_db)):
    return {"message": "Database dependency supplied"}
```

Dependencies can depend on other dependencies. Keep them focused, reusable and easy to test.

## 10. Authentication and authorization

**Authentication** answers “Who are you?” **Authorization** answers “What are you allowed to do?”

### Common approaches

• Session cookie: browser stores a secure session identifier; server manages session state.

• Bearer token: client sends Authorization: Bearer on each request.

• JWT: a signed token format that can carry claims; signing does not automatically encrypt the token.

• OAuth2 / OpenID Connect: standards commonly used for delegated authorization and identity.

• API key: simple credential often used for service-to-service access; protect and rotate it.

### Security essentials

• Hash passwords with a password-hashing algorithm such as Argon2 or bcrypt; never store plaintext passwords.

• Use HTTPS in deployed environments.

• Validate permissions on the server for every protected resource.

• Do not put secrets in frontend code, URLs, logs or Git repositories.

• Set token expiry and a revocation/refresh strategy appropriate to the application.

• Apply rate limits, input limits and careful CORS configuration.

A valid token does not mean the user may access every record. Check ownership and role permissions for the specific resource.

## 11. Error handling and validation

```
from fastapi import HTTPException

@app.get("/students/{student_id}")
def get_student(student_id: int):
    student = None  # Replace with a database lookup
    if student is None:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )
    return student
```

Use meaningful HTTP status codes and stable error structures. Avoid returning raw stack traces, SQL errors, passwords or internal implementation details to clients.

### Validation layers

• Request validation: required fields, types, length and format.

• Business validation: rules such as course capacity must not be exceeded.

• Database constraints: unique values, foreign keys and non-null constraints.

• Authorization: caller is permitted to perform this operation.

A request can be syntactically valid but violate a business rule. Keep these checks distinct.

## 12. Async, concurrency and performance

Use async def when the route performs awaitable I/O, such as calls to an async database driver or HTTP client. Use ordinary def for blocking libraries unless you deliberately offload blocking work.

```
import httpx
from fastapi import FastAPI

app = FastAPI()

@app.get("/external-status")
async def external_status():
    async with httpx.AsyncClient(timeout=10) as client:
        response = await client.get("https://example.com")
    return {"status_code": response.status_code}
```

Async improves how a server handles concurrent I/O waits; it does not make CPU-heavy Python code automatically faster. For CPU-intensive work, consider worker processes, task queues or separate compute services.

### Performance practices

• Use database indexes based on query patterns and inspect slow queries.

• Paginate large result sets; do not return unlimited records.

• Reuse HTTP clients and configure timeouts.

• Cache only data with a clear invalidation strategy.

• Measure latency and throughput before optimizing.

# 13. Project structure for a growing application

```
app/
  main.py
  core/
    config.py
    security.py
  api/
    routes/
      courses.py
      users.py
  schemas/
    course.py
    user.py
  models/
    course.py
    user.py
  services/
    course_service.py
  repositories/
    course_repository.py
  db/
    session.py
  tests/
    test_courses.py
.env
.gitignore
requirements.txt
```

A practical separation of responsibilities:

• Routes: HTTP layer; parse requests and return responses.

• Schemas: request and response data contracts.

• Models: database table mappings.

• Services: business rules and use cases.

• Repositories: database query details, when this abstraction is useful.

• Core/config: settings, security and shared application configuration.

Do not add layers just for the sake of folders. Introduce abstractions when they clarify responsibilities or make testing easier.

# 14. Testing APIs

Test both successful behavior and failure cases. FastAPI's TestClient can exercise the application without manually starting a server.

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "healthy"}
```

## What to test

• Correct status code and response body.

• Missing fields, invalid types and boundary values.

• Unknown resource IDs and duplicate records.

• Authentication and role/ownership permissions.

• Database rollback and transaction behavior.

• Pagination, filtering and sorting.

Use isolated test databases or controlled fixtures. Never let tests accidentally modify production data.

# 15. API documentation and versioning

FastAPI generates an OpenAPI schema from route declarations, type hints and Pydantic models. Swagger UI is commonly available at /docs and ReDoc at /redoc.

For breaking API changes, version the contract, for example /api/v1/courses. Keep old versions during a planned migration period when clients need time to upgrade.

## Good API design

• Use nouns for resource paths: /users/42/orders rather than action-heavy names.

• Use plural collection names consistently.

• Return predictable response shapes and status codes.

• Document query parameters, validation rules and error responses.

• Define pagination conventions, filtering and sorting consistently.

# 16. CORS, deployment and DevOps basics

CORS (Cross-Origin Resource Sharing) is a browser security mechanism. If a frontend at one origin calls an API at another origin, the backend may need to allow that frontend origin.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5500"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

For production, specify trusted origins instead of using a wildcard, especially when credentials are enabled.

## Typical deployment flow

• Developer writes code and tests locally.

• Code is committed and pushed to Git.

• CI pipeline installs dependencies, runs linting and tests.

• Build a Docker image if the team uses containers.

• Publish the image to a container registry.

• Deployment system releases the approved image to staging/production.

• Monitor logs, metrics, traces and health checks; roll back if necessary.

A production service also needs environment-specific configuration, secret management, database migrations, TLS termination, backups and an operational rollback plan.

# 17. Practical project: Learning App API

Build a backend for a learning application. Start small and add one feature at a time.

| Feature | Endpoints to implement |
|---|---|
| Health | GET /health |
| Courses | POST /courses, GET /courses, GET /courses/{id} |
| Course maintenance | PUT/PATCH /courses/{id}, DELETE /courses/{id} |
| Students | POST /students, GET /students/{id} |
| Enrollment | POST /courses/{id}/enrollments |
| Progress | PATCH /students/{id}/progress |
| Authentication | POST /auth/register, POST /auth/login |
| Admin | Protected course create/update/delete routes |

## Suggested database tables

| Table | Important columns |
|---|---|
| users | id, name, email (unique), password_hash, role |
| courses | id, title, description, level, created_at |
| enrollments | id, user_id, course_id, enrolled_at, status |
| progress | id, enrollment_id, lesson_id, completed_at |

Define foreign keys and unique constraints. For example, a user should not have duplicate active enrollment in the same course. Decide whether this rule belongs in a unique database constraint, application logic, or both.

## Build sequence

• Day 1: Python environment, FastAPI app, routes and Swagger docs.

• Day 2: path/query/body parameters and Pydantic validation.

• Day 3: CRUD endpoints using temporary in-memory data.

• Day 4: MySQL setup and SQLAlchemy models/sessions.

• Day 5: CRUD with database persistence and migrations.

• Day 6: authentication, password hashing and authorization.

• Day 7: tests, error handling, pagination and clean project structure.

• After Day 7: Docker, CI pipeline, deployment and monitoring.

# 18. Common beginner mistakes

• Confusing REST (design style) with FastAPI (framework).

• Using GET requests to change or delete data.

• Putting all business logic directly inside route functions.

• Trusting frontend validation without validating on the backend.

• Returning 200 for every outcome instead of meaningful status codes.

• Storing passwords or database credentials in plaintext.

• Using a global database session shared across requests.

• Using async def with blocking calls and assuming it is automatically faster.

• Returning every database row without pagination.

• Allowing all CORS origins in production without a reason.

• Testing only the happy path.

# 19. Interview questions with short answers

**Q. What is an API?**
A defined interface through which software components communicate.

**Q. What is REST?**
An architectural style for network APIs based on resources and constraints.

**Q. What is an endpoint?**
A URL path and method combination that exposes an API operation.

**Q. What is FastAPI?**
A Python framework for building APIs with type-driven validation and OpenAPI support.

**Q. FastAPI vs REST API?**
FastAPI is a framework; REST is an API architectural style. FastAPI can build REST APIs.

**Q. GET vs POST?**
GET retrieves data; POST commonly creates a resource or triggers processing.

**Q. PUT vs PATCH?**
PUT generally replaces a resource representation; PATCH applies a partial update.

**Q. 401 vs 403?**
401 means valid authentication is missing or failed; 403 means access is not permitted.

**Q. What is Pydantic?**
A library for parsing and validating data using Python type annotations.

**Q. What is dependency injection?**
A pattern where required components are supplied to a function rather than created inside it.

**Q. What is SQLAlchemy?**
A Python SQL toolkit and ORM for database access.

**Q. What is a migration?**
A versioned change to database schema.

**Q. What is CORS?**
A browser mechanism controlling cross-origin access to resources.

**Q. What is JWT?**
A compact token format that can carry signed claims; it is not automatically encrypted.

**Q. What is idempotency?**
Repeating an operation has the same intended effect as performing it once.

**Q. Why use response_model?**
To define, validate and document the response shape and avoid exposing unwanted fields.

**Q. When use async def?**
When using awaitable I/O operations; blocking work needs separate handling.

**Q. What is OpenAPI?**
A standard description format for HTTP APIs, used to generate docs and client tooling.

# 20. Quick revision checklist

• Explain API, REST, resource, endpoint, request and response.

• Know GET, POST, PUT, PATCH and DELETE.

• Know 200, 201, 204, 400, 401, 403, 404, 409, 422 and 500.

• Create FastAPI routes and use path/query/body parameters.

• Create Pydantic request and response models.

• Build CRUD endpoints and connect a database with SQLAlchemy.

• Use Depends for database sessions and reusable checks.

• Explain authentication, authorization, password hashing and JWT risks.

• Write tests for success, validation and authorization failures.

• Explain CORS, environment variables, Docker and CI/CD at a high level.

# Recommended official references

Use the official documentation as the source of truth because framework APIs and recommended patterns can change:

• FastAPI documentation

• MDN HTTP guide

• RFC 9110 — HTTP Semantics

• OpenAPI Specification

• Pydantic documentation

• SQLAlchemy 2.0 documentation

Study tip: Read one section, implement its example, call the endpoint from /docs or an API client, then deliberately send invalid input and inspect the response.